{EPITECH}

DAY 13_

< LET'S END THIS, KID. />

DAY 13

binary name: main
language: Rust

- ✓ The totality of your source files, except all useless files (binary, temp files, obj files,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.

{EPITECH}

Exercise 00 - Ship Characteristics

Turn-in : `ship_characteristics.rs` in a folder named `ex00`

You have been moved to the security team because of a lack of human resources. Security is very important for The Union ! You task is to oversee a lot of things. Each component must have the right behavior !

Write a bunch of implementations for these structs based on the traits below. You must implement:
- `RocketComponent` for each struct
- `SafetyCheck` for `RocketBooster`, `FuelTank` and `CrewCabin`
- `LoadContent` for `FuelTank` and `SciencePayload`

```
pub struct RocketBooster {
    height: f32,
    diameter: f32,
    last_check_time: u32,
}

pub struct FuelTank {
    height: f32,
    quantity: f32,
    max_quantity: f32,
    last_check_time: u32,
}

pub struct CrewCabin {
    nb_seats: u8,
    diameter: f32,
    last_check_time: u32,
}

pub struct SciencePayload {
    quantity: f32,
    max_quantity: f32,
    science_percentage: f32,
}

pub trait RocketComponent {
    fn new() -> Self;
    fn info(&self) -> String;
}

pub trait SafetyCheck {
    fn check_component(&mut self, current_time: u32) -> bool;
    fn is_safe(&self, current_time: u32) -> bool;
}

pub trait LoadContent {
    fn load(&mut self, to_load: f32);
    fn is_full(&self) -> bool;
}
```


{EPITECH}

Concerning the differents functions you will have to write, here are some informations:
- The `last_check_time` of a component is the day it was last checked since its creation. So a component with a `last_check_time` of 30 was checked for the last time 30 days after its creation. (If we're on the 31st day after its creation day it would probably be okay, but if we're on the 89th day after its creation it would probably be really unsafe to use it since it has not been verified for 59 days).
- A `RocketBooster` is 32.3 meters tall, has a diameter of 3 meters and needs to be checked for safety reasons every 12 days.
- A `FuelTank` is 30 meters tall, needs to be checked for safety reasons every 6 days, is obviously empty at construction and can contain 220 tons of propellant (fuel).
- The `CrewCabin` has four seats, has a diameter of 2.5 meters and needs to be checked every 16 days.
- The `SciencePayload` is contructed empty but can contain up to 1.142 tons of payload. The Union is very concerned about science, so for each percent of the payload effectively filled you should double it to count the science percentage of the payload. So if the payload contains 685.2kg of science material, the science percentage is going to be 120%. But the payload is not full, you can load more science material, there's just a lot of science...
- The `new` function initialize a struct with the informations given above, print some informations and return it (examples below).
- The `info` function print informations about the component (examples below).
- The `check_component` function is called when safety tests are run on the component and it resets its last day checked to the one passed as parameter. If the day given as parameter is lower, the operator probably did something wrong, in that case, return false and don't change the date.
- The `is_safe` function return true if the number of days since the last check is lower than the maximum number of days between to check (written above for each component) and false otherwise. If the day given as parameter is lower, the operator probably did something wrong, in that case, return false. A component is considered unsafe if its last_check_time is zero.
- The `load` function loads the container with the number given as parameter. The container can never have more quantity than the maximum it can carry.

AND FINALLY...

For the `SciencePayload` you must implement another method called `science` that takes no parameters and returns the percentage of science of the payload but also brags about science discovery of The Union listed in the module like this:

```
pub const DISCOVERIES: [&str; 4] = [
    "The Union discovered that the hottest planet in our solar system is Venus (about 450
        °C).",
    "The Union discovered that Mercury and Venus are the only planets in our solar system
        with no moons.",
    "The Union discovered that one day on Venus is longer than a year on Earth.",
    "The Union discovered that on Venus it snow metal and rains sulfuric acid."
];
```

If the percentage of science is between 0 (included) and 50 (excluded) print the first one on screen; between 50 (included) and 100 (excluded) print the second one; between 100 (included) and 150 (excluded) print the third one and between 150 (included) and 200 (included) print the fourth one.

```rust
fn main() {
    let mut booster = RocketBooster::new();
    let mut tank = FuelTank::new();
    let mut science = SciencePayload::new();
    let mut cabin = CrewCabin::new();

    println!("\n- {}\n- {}\n- {}\n- {}", booster.info(), tank.info(), science.info(),
        cabin.info());

    println!("Is the booster safe on day 4: {}", booster.is_safe(4));
    println!("Check the booster on day 6: {}", booster.check_component(6));
    println!("Is the booster safe on day 18: {}", booster.is_safe(18));
    println!("Is the booster safe on day 19: {}", booster.is_safe(19));

    tank.load(10.0);
    tank.check_component(9);
    println!("Is fuel tank safe: {}, is fuel tank full: {}.", tank.is_safe(10), tank.is_
        full());
    println!("Check tank on day 1: {}", tank.check_component(1));
    println!("Is tank safe on day 8: {}", tank.is_safe(8));

    science.load(140.0);
    println!("Science quantity: {:.02}", science.science());
    science.load(250.0);
    science.load(250.0);
    println!("Science quantity: {:.02}", science.science());

    println!("Is the cabin safe on day 5: {}", cabin.is_safe(5));
    println!("Check the cabin on day 6: {}", cabin.check_component(6));
    println!("Is the cabin safe on day 22: {}", cabin.is_safe(22));

    println!("\n- {}\n- {}\n- {}\n- {}", booster.info(), tank.info(), science.info(),
        cabin.info());
}
```

Terminal

```
$> ./my_ship_characteristics
Rocket Booster created with a height of 32.3 meters.
Fuel Tank created with a max quantity of 220 tons.
Science Payload created with a max quantity of 1142 kilos.
Cabin of 4 person created.
- This Rocket Booster is 32.3 meters tall and 3 meters wide. It was last checked on day 0
of its life.
- This Fuel Tank is 30 meters tall and can contain up to 220 tons of propergol, currently 0
tons inside. It was last checked on day 0 of its life.
- This Science Payload can carry 1142.00 kilos of science material. Currently it contains
0.00 kilos which means it has a science percentage of 0.00%.
- This cabin has 4 seats arranged in a circle 2.50 meters large. It was last checked on day
0 of its life.
Is the booster safe on day 4: false
Check the booster on day 6: true
Is the booster safe on day 18: true
Is the booster safe on day 19: false
Is fuel tank safe: true, is fuel tank full: false.
Check tank on day 1: false
Is tank safe on day 8: false
The Union discovered that the hottest planet in our solar system is Venus (about 450°C).
Science quantity: 24.52
The Union discovered that one day on Venus is longer than a year on Earth.
Science quantity: 112.08
Is the cabin safe on day 5: false
Check the cabin on day 6: true
Is the cabin safe on day 22: true
- This Rocket Booster is 32.3 meters tall and 3 meters wide. It was last checked on day 6
of its life.
- This Fuel Tank is 30 meters tall and can contain up to 220 tons of propergol, currently
10 tons inside. It was last checked on day 9 of its life.
- This Science Payload can carry 1142.00 kilos of science material. Currently it contains
640.00 kilos which means it has a science percentage of 112.08%.
- This cabin has 4 seats arranged in a circle 2.50 meters large. It was last checked on day
6 of its life
```

Exercise 01 - Mission Report

Turn-in : `mission_report.rs` in a folder named `ex01`

The Union is undergoing a massive bureaucratic update. The High Command has mandated that all Mission Control software must adhere to the new **Standard Intergalactic Protocol (SIP)**.

Your code is currently non-compliant. You defined structures, but they don't integrate with the ecosystem. The High Command wants your objects to behave like native Rust types.

You have three goals to achieve to certify your module.

The Data Structures

Start with these bare definitions. Do not modify the fields.

```
pub enum MissionState {
    Planned,
    InProgress,
    Completed,
    Failed,
}

pub struct Rover {
    pub name: String,
    pub fuel_level: u8,
    pub map_sectors: Vec<u32>,
}

pub struct Mission {
    pub mission_name: String,
    pub rover: Rover,
    pub state: MissionState,
}
```

1. The Engineer's Toolkit

Engineers work in the simulation lab. They have specific needs for debugging and prototyping:

1. They need to be able to inspect **any** of these structures (Enums and Structs) in the logs using the standard debug formatter (`{:?}`).
2. They often run simulations by duplicating **Rovers**. Your code must allow a `Rover` to be explicitly duplicated (cloned).
3. However, a **Mission** is a unique bureaucratic instance. It is strictly **forbidden** to clone a Mission. The compiler must prevent this.
4. Engineers need to compare two **Rovers** to check if they are strictly identical (same name, same fuel, same map).

Use the most efficient way to generate this code.

2. The Director's Report

The Director doesn't look at debug logs. He reads official reports printed on paper.
When a `Mission` object is printed using the user-facing formatter (`"{}"`), it must output a clean, formatted block of text.

The required format is:

```
[<STATE>] <mission_name>
  > Assigned Rover: <rover_name> (Fuel: <fuel_level>%)
  > Map Coverage: <count> sectors
```

Constraints:

✓ The `<STATE>` must be printed in uppercase (e.g., `[IN PROGRESS]`, `[FAILED]`).
✓ The logic must be modular: The `Mission` shouldn't know how to format a `Rover` or a `State`. Each structure should be responsible for its own display format.

{EPITECH}

3. The Deduplication Protocol

The database is corrupted and contains duplicate entries. We need to clean it up.
You must enable the use of the `==` operator between two `Mission` objects.

> Two missions are considered ”Equal” if and only if they share the **same name**, regardless of their state or assigned rover.

4. The Priority Queue

The High Command receives too many reports. They need to sort them by **Urgency**.
You must ensure that a `Vec<MissionState>` can be sorted using the standard `sort()` method.

The priority order (highest urgency to lowest):

1. **Failed** (requires immediate rescue)
2. **InProgress** (requires monitoring)
3. **Planned**
4. **Completed** (can be archived)

Example

The following code must compile and run without errors.

```
fn main() {
    let r1 = Rover { name: "Spirit".to_string(), fuel_level: 0, map_sectors: vec![] };
    let r2 = Rover { name: "Opportunity".to_string(), fuel_level: 50, map_sectors: vec![1,
        2] };
    let r3 = Rover { name: "Curiosity".to_string(), fuel_level: 100, map_sectors: vec![1,
        2, 3, 4] };
    let m1 = Mission { mission_name: "Alpha".to_string(), rover: r1.clone(), state:
        MissionState::Failed };
    let m2 = Mission { mission_name: "Alpha".to_string(), rover: r2.clone(), state:
        MissionState::InProgress };
    let m3 = Mission { mission_name: "Beta".to_string(),  rover: r3.clone(), state:
        MissionState::Completed };
    println!("--- Report ---");
    println!("{}", m1);
    println!("\n--- Deduplication ---");
    if m1 == m2 {
        println!("Mission '{}' is a duplicate of Mission '{}'.", m1.mission_name, m2.
            mission_name);
    } else {
        println!("Missions are different.");
    }
    println!("\n--- Priority Sorting ---");
    let mut states = vec![
        MissionState::Planned,
        MissionState::Failed,
        MissionState::Completed,
        MissionState::InProgress,
    ];
    states.sort_by(|a, b| b.cmp(a));
    for s in states {
        println!("{:?}", s);
    }
}
```

Terminal

```
$> cargo run
--- Report ---
[FAILED] Alpha
Assigned Rover: Spirit (Fuel: 0%)
Map Coverage: 0 sectors
--- Deduplication ---
Mission 'Alpha' is a duplicate of Mission 'Alpha'.
--- Priority Sorting ---
Failed
InProgress
Planned
Completed
```


{EPITECH}

Exercise 02 - Universal Storage

Turn-in : `universal_storage.rs` in a folder named `ex02`

Logistics Officer's Log, Stardate 42.1:
We are wasting too much metal. Until now, we had `FoodBox` for food, `WeaponBox` for weapons, and `FuelBox` for fuel. It's inefficient.
The R&D department has finally developed a **Universal Container**. It can hold *anything*, regardless of its atomic structure.

Your mission is to implement this standardized container using **Generics**.

Define a structure named `Storage` that takes a generic type parameter `T`.
It must have a single public field named `item` of type `T`.

```
pub struct Storage<T> {
    // ...
}
```

Implement the following two methods for `Storage<T>`.

`new` creates a new generic storage container holding the given item.

```
pub fn new(item: T) -> Storage<T>
```

`extract` takes ownership of the storage, destroys the box, and returns the item inside.

```
pub fn extract(self) -> T
```

Example

```
fn main() {
    let number_box = Storage::new(42);
    let number = number_box.extract();
    println!("Extracted number: {}", number);

    let string_box = Storage::new(String::from("Nano-Material"));
    let material = string_box.extract();
    println!("Extracted material: {}", material);
}
```


{EPITECH}

```
Terminal
$> cargo run
Extracted number: 42
Extracted material: Nano-Material
```

Exercise 03 - The Best Candidate

Turn-in : `best_candidate.rs` in a folder named `ex03`

The Union's Human Resources department is overwhelmed. We have lists of candidates for the Venus Mission, but the data comes in different formats depending on the department sending it.

- ✓ The Physical Team sends lists of **integers** (scores out of 100).
- ✓ The Science Team sends lists of **floats** (precision rating).
- ✓ The Administration sends lists of **names** (alphabetical sorting).

The HR Director wants a **single, universal tool** to identify the ”best” (highest) element in any list, regardless of the data type.

We tried to write a generic function, but the compiler started screaming about ”binary operations”. Your job is to fix it.

Instructions

Create a function named `winning_candidate` that:

1. Accepts a slice (`&[T]`) of **any type** `T`.
2. Iterates through the list to find the element with the **highest value**.
3. Returns an `Option` containing a reference to that element (`Option<&T>`).

Functional Rules:

- ✓ If the list is empty, return `None`.
- ✓ If the list has content, return `Some(max_value)`.
- ✓ ”Highest” means the value that is strictly greater (`>`) than the others.

You must **restrict** the generic type `T` to only accept types that support comparison operations.

- ✓ You are **FORBIDDEN** to duplicate code (do not write one function for `i32` and one for `f32`).
- ✓ You are **FORBIDDEN** to use `.max()` or `.sort()` from the standard library. You must implement the comparison logic loop manually to prove you understand how to compare `T`.

Example

Your solution must allow this `main` to run perfectly:

```rust
fn main() {
    let scores = vec![75, 92, 88, 100, 42];
    match winning_candidate(&scores) {
        Some(v) => println!("Highest Score: {}", v),
        None => println!("No scores provided"),
    }

    let precision = vec![0.12, 5.9, 0.01, 3.14];
    match winning_candidate(&precision) {
        Some(v) => println!("Highest Precision: {}", v),
        None => println!("No precision data"),
    }

    let candidates = vec![
        String::from("Amelia"),
        String::from("Yuri"),
        String::from("Neil"),
        String::from("Buzz"),
    ];
    match winning_candidate(&candidates) {
        Some(v) => println!("Last Alphabetical Name: {}", v),
        None => println!("No candidates"),
    }

    let empty_list: Vec<i32> = vec![];
    match winning_candidate(&empty_list) {
        Some(_) => println!("This should not print"),
        None => println!("Correctly handled empty list"),
    }
}
```

```
Terminal
$> cargo run
Highest Score: 100
Highest Precision: 5.9
Last Alphabetical Name: Yuri
Correctly handled empty list
```


{EPITECH}

Exercise 04 - Temperature Analysis

Turn-in : `temp_analysis.rs` in a folder named `ex04`

The Environmental Control System sends a continuous stream of temperature readings from the reactor core. However, the sensors are cheap. They sometimes send impossible negative values (absolute zero violations) or fluctuations that need to be smoothed out.

You need to clean this data stream using **Functional Programming**.

Flashbacks?
If this reminds you of your struggles with **Haskell** during the start of this pool, don't panic. Rust embraces Functional Programming patterns.
Think like a Haskeller: No loops, just data flowing through a pipeline of functions.

Implement the function `calculate_average_temperature`:

```
pub fn calculate_average_temperature(readings: &[f64]) -> Option<f64>
```

Your pipeline must follow these steps **in order**:

1. **Filter**: Ignore any value strictly below `-273.15` (Physical impossibility).
2. **Map**: The sensors are uncalibrated. Add `+2.0` degrees to every remaining value.
3. **Aggregate**: Calculate the **Average** (Mean) of the valid readings.

Return:

- ✓ `Some(average)` if there are valid readings.
- ✓ `None` if the list is empty or all values were filtered out.

Forbidden:

- ✓ `for`, `while`, `loop`.
- ✓ Mutable variables (`mut`) outside of the result calculation.

{EPITECH}

Example

```rust
fn main() {
    let readings = vec![
        23.0,
        -300.0,
        20.5,
        -273.15
    ];

    match calculate_average_temperature(&readings) {
        Some(avg) => println!("Average Temp: {:.2}", avg),
        None => println!("No valid readings"),
    }
}
```

```
Terminal
$> cargo run
Average Temp: -74.55
```


{EPITECH}

Exercise 05 - Parallel Dimensions

Turn-in : `parallel_dims.rs` in a folder named `ex05`

You are receiving telemetry from two parallel dimensions: the **Dimension of Hot Strings** and the **Dimension of Fast Integers**.
Your job is to synchronize these two timelines to calculate the total energy potential of the universe.

The problem? Both dimensions are noisy.

1. **Hot Strings**: Contains temperature values, but some are garbled text (”error”, ”NaN”, garbage).
2. **Fast Integers**: Contains engine rotation speeds, but we only care about engines running at **high velocity** (> 2500).

Instructions

Implement the function `fusion_processing`:

```
pub struct Report {
    pub sample_count: usize,
    pub total_energy: f64,
}

pub fn fusion_processing(temp_log: &[&str], rpm_log: &[u32]) -> Report
```

The Rules of Physics

Your function must produce a `Report` based on the following laws:

1. **Data Purity:** You must purge the data before using it.

✓ Any temperature string that cannot be parsed into a float is cosmic dust: **discard it**.
✓ Any RPM strictly below or equal to 2500 is idle noise: **discard it**.

2. **The Synchronization:**

✓ The logs are sequential. Once the noise is removed from both sides, the **1st valid** temperature corresponds to the **1st valid** RPM. The 2nd matches the 2nd, and so on.

{EPITECH}

✓ If one dimension has more valid data points than the other, the excess data is lost in the void (stop processing).

3. **The Energy Formula:**

✓ For each synchronized pair, `Energy = Temperature / RPM`.
✓ **Safety Cap:** If the calculated Energy is infinite or impossible (e.g., extremely high), cap it at `100.0`.

4. **The Output:**

✓ `sample_count`: How many pairs were successfully fused.
✓ `total_energy`: The sum of all calculated energies.

Hard Constraints:

✓ **NO LOOPS.** No `for`, `while`, or `loop` keywords.
✓ **NO MUTABLE VARIABLES** declared outside of your final aggregator.
✓ Think functional. If you are writing more than one semicolon, you are probably doing it wrong (though not strictly forbidden, try to chain it).

Remember your Haskell nightmares? Good. You will need that mindset. Do not try to iterate manually. Think of this as two streams of water merging into one pipe. Rust's Iterator trait has all the tools you need to clean, merge, and consolidate these streams.

Example

```rust
fn main() {
    let temps = vec![
        "30.0",
        "garbage",
        "100.0",
        "50.0"
    ];

    let rpms = vec![
        1000,
        3000,
        5000,
    ];

    let report = fusion_processing(&temps, &rpms);

    println!("Fused {} pairs. Total Energy: {}", report.sample_count, report.total_energy)
        ;
}
```

```
Terminal
$> cargo run
Fused 2 pairs. Total Energy: 0.03
```

Exercise 06 - Life on Venus

Turn-in : `venus.rs` in a folder named `ex06`

The Union would like you to simulate what it would be to encounter bacterial life on Venus. From our perspective it would look like a game with no player. A game for life itself... a Game of Life.

Given this struct and this trait in a `life_struct.rs`:

```
mod life_struct {
    #[derive(Default)]
    pub struct Board {
        pub(super) size_x: i64,
        pub(super) size_y: i64,
        pub(super) board: Vec<char>,
    }

    pub trait GameOfLife {
        fn new(size_x: i64, size_y: i64) -> Self;
        fn next(&mut self);
        fn add_life(&mut self, x: i64, y: i64) -> Result<(), (String)>;
    }
}
```

Think about how you can use theses items from our `life_struct.rs` file.
Mhhh... crate ? Crate ! Crate !!

Implment everything needed to make this main work:

```
fn main() {
```


{EPITECH}

```
    let mut c: Board = Board::new(10, 10);
    let pos: [(i64, i64); 6] = [(2, 6), (1, 7), (3, 6), (3, 7), (3, 8), (3, 10)];
    let mut i: i32 = 0;

    for (x, y) in pos {
        match c.add_life(x, y) {
            Ok(()) => (),
            Err(x) => println!("{x}"),
        };
    }
    while i < 3 {
        println!("{c}\n");
        c.next();
        i += 1;
    }
}
```

```
Terminal
$> Wrong index
..........
..........
..........
..........
..........
..........
..oo......
.o.o......
...o......
..........
..........
..........
..........
..........
..........
..........
..oo......
...oo.....
..o.......
..........
..........
..........
..........
..........
..........
..........
..ooo.....
....o.....
...o......
..........
```

You will definitely have to implement something needed but not mentionned for this `main` function to work, but you should find it quite easily.

{EPITECH}

v1